

Multimodal Visual Understanding Application

Multimodal Visual Understanding Application

1. Concept Introduction
 - 1.1 What is "Visual Understanding"?
 - 1.2 Brief Description of the Principle
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largetmodel/utis/tools_manager.py`)
 2. Model Interface Layer (`largetmodel/utis/large_model_interface.py`)
 - Code Analysis
3. Practical Operation
 - 3.1 Configure Offline Large Model
 - 3.1.1 Configure LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configure Model Interface (`large_model_interface.yaml`)
 - 3.2 Start and Test the Function (Text Input Mode)
4. Common Problems and Solutions
 - Problem 1: The log shows "Failed to call ollama vision model" or the connection is refused.
 - Problem 2: The `seewhat` tool returns "Unable to open camera" or fails to take a picture.

Note: The RDK X5 4GB version cannot run this offline due to performance limitations. Please refer to the tutorial for online large models.

1. Concept Introduction

1.1 What is "Visual Understanding"?

In the `largetmodel` project, the **multimodal visual understanding** function means allowing the robot not just to "see" a pixel matrix, but to truly "understand" the content, objects, scenes, and their relationships within an image. It's like giving the robot a pair of thinking eyes.

The core tool for this function is `seewhat`. When the user issues a command like "see what's here," the system calls this tool, triggering a series of background operations that ultimately provide the user with the AI's analysis of the real-time scene in natural language.

1.2 Brief Description of the Principle

The basic principle is to input two different types of information—**images (visual information)** and **text (language information)**—into a powerful multimodal large model (such as LLaVA).

1. **Image Encoding:** The model first converts the input image into numerical vectors that the computer can understand using a Vision Encoder. These vectors capture the image's color, shape, texture, and other features.
2. **Text Encoding:** At the same time, the user's question (e.g., "What's on the table?") is also converted into a text vector.

3. **Cross-modal Fusion:** In the most critical step, the model fuses the image and text vectors in a special "Attention Layer." Here, the model learns to "focus" on the parts of the image relevant to the question. For example, when asked about the "table," the model will pay more attention to the area in the image that matches the features of a "table."
4. **Answer Generation:** Finally, a large language model (LLM) section generates a descriptive text as the answer based on the fused information.

In simple terms, it's about **using text to "highlight" the corresponding parts of the image, and then using language to describe the "highlighted" parts.**

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utis/tools_manager.py`)

The `seewhat` function in this file defines the execution flow of the tool.

```
# From largemodel/utis/tools_manager.py

class ToolsManager:
    # ...

    def seewhat(self):
        """
        Capture camera frame and analyze environment with AI model.

        :return: Dictionary with scene description and image path, or None if
        failed.
        """
        self.node.get_logger().info("Executing seewhat() tool")
        image_path = self.capture_frame()
        if image_path:
            # Use isolated context for image analysis.
            analysis_text = self._get_actual_scene_description(image_path)

            # Return structured data for the tool chain.
            return {
                "description": analysis_text,
                "image_path": image_path
            }
        else:
            # ... (Error handling)
            return None

    def _get_actual_scene_description(self, image_path, message_context=None):
        """
        Get AI-generated scene description for captured image.

        :param image_path: Path to captured image file.
        :return: Plain text description of scene.
        """
        try:
```

```

        # ... (Build Prompt)
        result = self.node.model_client.infer_with_image(image_path,
scene_prompt, message=simple_context)
        # ... (Process result)
        return description
    except Exception as e:
        # ...

```

2. Model Interface Layer

(`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image understanding tasks. It is responsible for calling the specific model implementation based on the configuration.

```

# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
        # ... (Prepare message)
        try:
            # Decide which specific implementation to call based on the value of
            self.llm_platform
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Logic for calling Tongyi model
                pass
            # ... (Logic for other platforms)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}

```

Code Analysis

The implementation of this function involves two main layers: the tool layer defines the business logic, and the model interface layer is responsible for communicating with the large language model. This layered design is key to achieving platform versatility.

1. Tool Layer (`tools_manager.py`):

- The `seewhat` function is the business core of the visual understanding function. It encapsulates the complete flow of the "see" action: first, it calls the `capture_frame` method to get an image, then it calls `_get_actual_scene_description` to prepare a prompt for requesting the model to analyze the image.
- The most critical step is calling the `infer_with_image` method of the model interface layer. It doesn't care which underlying model is used; it is only responsible for passing the core data of "image" and "analysis instruction."
- Finally, it packages the analysis result (plain text description) received from the model interface layer into a structured dictionary and returns it. This allows the upper-level application to use the analysis result conveniently.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function acts as a "dispatch center." Its main responsibility is to check the current platform configuration (`self.llm_platform`) and dispatch the task to the corresponding specific processing function (e.g., `ollama_infer` or `tongyi_infer`) based on the configuration value.
- This layer is the key to adapting to different AI platforms. All platform-specific operations (such as data encoding, API call formats, etc.) are encapsulated in their respective processing functions.
- In this way, the business logic code in `tools_manager.py` can support multiple different backend large model services without any changes. It only needs to interact with the unified and stable `infer_with_image` interface.

In summary, the execution flow of the `seewhat` tool reflects a clear separation of responsibilities pattern: `ToolsManager` is responsible for defining "what to do" (get an image and request analysis), while `model_interface` is responsible for defining "how to do it" (select the appropriate model platform according to the current configuration and complete the interaction with it). This makes the tutorial's analysis universal, and its core code logic is consistent whether the user is in online or offline mode.

3. Practical Operation

3.1 Configure Offline Large Model

3.1.1 Configure LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its main language model.

1. Open the file in the terminal:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. Modify/Confirm `llm_platform`:

```
model_service:                                # Model server node parameters
  ros__parameters:
    language: 'en'                             # Large model interface language
    useolinetts: True                          # This item is invalid in text mode
    and can be ignored

    # Large model configuration
    llm_platform: 'ollama'                     # Key: Make sure this is 'ollama'
    regional_setting : "China"
    camera_type: 'csi'                         # Camera type: 'csi', 'usb'
```

3.1.2 Configure Model Interface (`large_model_interface.yaml`)

This file defines which vision model to use specifically when the platform is selected as `ollama`.

1. Open the file in the terminal

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the ollama-related configuration

```
#.....  
## Offline Large Language Models  
# ollama configuration  
ollama_host: "http://localhost:11434" # ollama server address  
ollama_model: "llava" # Key: Change this to the multimodal model you  
have downloaded, such as "llava"  
#.....
```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

3.2 Start and Test the Function (Text Input Mode)

Note: The effect will be poor when using a model with a small number of parameters, or when the running memory is at its limit. For a better experience with this function, please refer to the corresponding chapter of <Online Large Model (Text Interaction)>.

1. **Start the `largemodel` main program (text mode):**

Open a terminal, enter the docker, and then run the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

2. **Send a text command:**

Open another new terminal, enter the same docker container, and run the following command:

```
ros2 run text_chat text_chat
```

Then start entering the text: "What do you see".

3. **Observe the results:**

In the first terminal running the main program, you will see log output showing that the system has received the text command, called the `seewhat` tool, and finally printed the text description of the desktop generated by the LLaVA model.

4. Common Problems and Solutions

Problem 1: The log shows "Failed to call ollama vision model" or the connection is refused.

Solution:

1. **Check the Ollama service:** Run `ollama list` in the terminal to ensure that `llava` (or the model you configured) has been downloaded and the Ollama service is running.
2. **Check the configuration file:** Carefully check whether the configurations in `yahboom.yaml` and `large_model_interface.yaml` are correct, especially the values of `llm_platform` and `ollama_model`.

Problem 2: The `seewhat` tool returns "Unable to open camera" or fails to take a picture.

Solution:

1. **Check the device connection:** Run `ls /dev/video*` to ensure that the system can detect your USB camera.
2. **Permission issues:** Try to test the camera with applications such as `cheese` or `gview`. If `sudo` can but ordinary users cannot, it may be a udev rule or user group permission issue.